

GF-DiT: Scheduling Parallelism for Diffusion Transformer Serving

Xinwei Qiang
qiangxinwei@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Jing Yang
jyang23@gzu.edu.cn
Guizhou University
Guiyang, China

Yu Feng
y-feng@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Yifan Hu
hu_y_f@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Han Zhao
zhao-han@cs.sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Jingwen Leng
leng-jw@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Shixuan Sun
sunshixuan@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Chen Chen
chen-chen@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Minyi Guo
guo-my@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Abstract

Diffusion Transformers (DiTs) have become the dominant architecture for image and video generation, creating growing demand for efficient DiT serving. Existing systems assign each request a fixed parallel configuration throughout its lifetime. However, DiT workloads exhibit substantial heterogeneity across requests, execution stages, and system conditions, making static parallelism inefficient and often leading to poor GPU utilization and degraded service quality.

This paper argues that DiT serving should treat GPU parallelism as a first-class schedulable resource. We present **GF-DiT**, a policy-programmable runtime for elastic DiT serving that dynamically adapts the parallelism of running requests according to workload demands and service objectives. GF-DiT introduces an asynchronous execution abstraction that decomposes requests into independently schedulable trajectory tasks and enables online GPU reallocation. To make elastic parallelism practical, GF-DiT further proposes group-free collectives, a lightweight communication abstraction that supports low-overhead online formation and reconfiguration of arbitrary execution groups.

We implement GF-DiT in vLLM-Omni and evaluate it on representative image and video diffusion workloads. Compared with fixed-pipeline execution with static parallelism, GF-DiT improves throughput by up to **6.01×**, reduces mean latency by up to **95%**, lowers SLO violation rates by up to **90%**, and reduces communication-group setup overhead from **778 ms** to approximately **60 μ s**.

Our code is available at <https://github.com/SJTU-Liquid/GF-DiT>.

1 Introduction

DiTs [27] have emerged as the dominant architecture for modern generative AI workloads, enabling high-quality text-to-image, text-to-video, and multimodal content generation [5, 11–13, 16, 19, 23, 28, 36, 39, 43, 46, 47, 49]. As demand for T2I and T2V services continues to grow, efficiently serving DiT models has become a critical systems challenge.

Unlike large language models (LLMs), which process requests through a prefill stage followed by autoregressive decoding, DiT inference consists of three stages: an *encoder* that transforms prompts into embeddings, a compute-intensive *denoising* stage that iteratively refines latent states through tens of diffusion steps, and a *decoder* that generates the final image or video. Among these stages, denoising dominates execution time and resource consumption.

To serve DiT workloads, recent systems such as vLLM-Omni [45] and SGLang Diffusion [31] adopt a fixed multi-GPU execution strategy. As illustrated in Fig. 1, each request is assigned a predetermined parallel configuration, and the encoder, denoising, and decoder stages are executed atomically under the same resource allocation. However, DiT workloads exhibit substantial heterogeneity. Across requests, output resolution, video duration, and the number of denoising steps vary significantly, resulting in orders-of-magnitude differences in computational demand. Within a request, different stages expose fundamentally different degrees of parallelism: encoder and decoder stages are lightweight, while denoising can effectively utilize substantially more GPUs.

Consequently, fixed-pipeline execution with static parallelism leads to two fundamental limitations. First, long-running requests introduce head-of-line (HoL) blocking, delaying shorter requests and degrading service quality. Second, static parallelism often leads to poor resource efficiency.

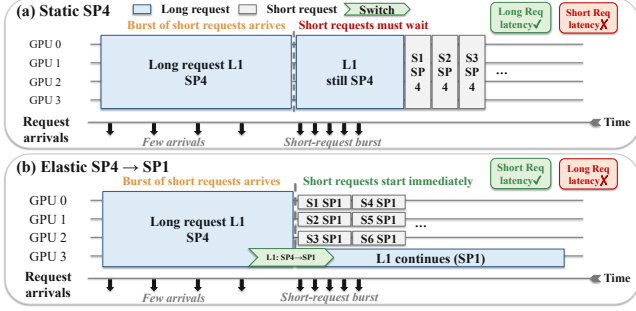


Figure 1. Elastic parallelism exposes policy tradeoffs. Static SP4 preserves the long request’s latency but makes short requests wait; shrinking it to SP1 admits short requests sooner at the cost of delaying the long request.

Lightweight stages and small requests cannot effectively utilize many GPUs, whereas insufficient parallelism prolongs large denoising workloads. As a result, existing systems struggle to simultaneously achieve high SLO attainment and high GPU utilization.

At the root of these limitations is the static nature of parallelism in existing DiT serving systems. Once a request is assigned a parallel configuration, its GPU allocation remains unchanged throughout execution. Consequently, the system cannot react to workload dynamics such as request arrivals, changing queue conditions, or evolving service objectives. A request that is appropriately provisioned when it begins execution may later become either over-provisioned or under-provisioned as system conditions evolve.

These observations motivate a different perspective: **instead of treating parallelism as a fixed execution configuration, a DiT serving system should treat parallelism as a first-class schedulable resource.** Similar to how operating systems dynamically allocate CPU time among competing processes, a DiT serving system should dynamically allocate GPU parallelism among competing requests according to workload demands and service objectives. Under this abstraction, GPU resources can continuously flow to where they provide the greatest benefit. In other words, existing systems schedule requests, whereas a DiT serving runtime should additionally schedule parallelism.

However, realizing this vision efficiently is challenging. Dynamic parallelism adaptation requires the runtime to repeatedly reconfigure resource allocations among concurrently executing requests while minimizing disruption to ongoing computation. Moreover, different deployments often optimize for different objectives, including latency, throughput, fairness, and cost efficiency. Therefore, the challenge is not only to enable efficient runtime reconfiguration, but also to provide a programmable substrate upon which diverse scheduling policies can be built.

Our Work. To address these challenges, we present **GF-DiT**, a policy-programmable runtime for elastic DiT serving. GF-DiT is enabled by two unique properties of DiT workloads. First, DiT execution maintains a *lightweight execution state*: the data exchanged between stages and denoising steps consists primarily of latent states and embeddings whose sizes are typically only several MBs to tens of MBs. Second, DiT requests exhibit a *predictable execution structure*: the denoising trajectory is largely determined before execution begins, enabling accurate estimation of execution costs and future resource demands. Together, these properties make DiT serving uniquely amenable to online parallelism adaptation, as requests can be reconfigured frequently while their future execution costs remain predictable.

Building upon these insights, GF-DiT consists of three key components. First, we design an *asynchronous execution abstraction and runtime model* that decomposes a DiT request into independently schedulable trajectory tasks, including encoding, decoding, and individual denoising steps. Each task boundary serves as an explicit rescheduling point, allowing the runtime to continuously adapt GPU allocations throughout request execution. By exposing rescheduling opportunities across the entire request lifetime, GF-DiT elevates parallelism from a static deployment decision to a runtime-managed resource.

Second, elastic parallelism fundamentally requires execution groups to be created and reconfigured online. To enable this capability, we introduce *group-free collectives*, a lightweight communication abstraction that eliminates expensive communicator construction and supports arbitrary GPU-group formation through lightweight logical descriptors. Group-free collectives make dynamic parallelism practical by reducing runtime reconfiguration overheads to a negligible level.

Finally, we develop a set of runtime optimizations that improve programmability and deployment efficiency, including layout-aware artifact migration and simulation-driven policy optimization. Combined with GF-DiT’s programmable policy interface, these techniques allow users to rapidly evaluate and deploy customized scheduling strategies while reusing the same runtime substrate.

We implement GF-DiT in vLLM-Omni and evaluate it on both image and video diffusion workloads. The same runtime substrate supports throughput-oriented, latency-oriented, SLO-aware, and custom scheduling policies through a unified policy interface. Compared with fixed-pipeline execution with static parallelism, GF-DiT improves throughput by up to **6.01×**, reduces mean latency by up to **95%**, lowers SLO violation rates by up to **90%**, and reduces communication-group setup overhead from up to **778 ms** to approximately **60 μ s**. In summary, this paper makes the following contributions:

- We introduce *parallelism as a first-class schedulable resource*, a new abstraction for adaptive DiT serving.

- We design **GF-DiT**, a policy-programmable runtime for elastic DiT serving that enables online GPU reallocation through asynchronous execution and reschedulable trajectory tasks.
- We propose *group-free collectives*, which make dynamic execution-group formation practical by eliminating expensive communicator construction.
- We implement GF-DiT and demonstrate significant performance improvements over existing fixed-pipeline execution with static parallelism.

2 Background and Motivation

This section presents the key observations that motivate GF-DiT. We first show that DiT workloads naturally expose rescheduling boundaries throughout execution. We then show that their execution structure is largely predictable before execution begins. Finally, we demonstrate that no single parallel configuration is optimal across stages, requests, and system conditions, motivating our central thesis: GPU parallelism should be treated as a first-class schedulable resource.

2.1 Rescheduling Boundaries

LLM and diffusion serving both execute requests iteratively, but they expose fundamentally different opportunities for runtime scheduling. In autoregressive LLM serving, each generated token produces a valid continuation state, while chunked prefill incrementally constructs reusable KV caches [2]. These token-sequence boundaries naturally provide opportunities for batching, preemption, and request scheduling.

DiT execution follows a different structure. During denoising, the model repeatedly refines a latent state along a *diffusion trajectory*, i.e., the sequence of latent states traversed from an initial noisy latent to the final clean latent. Within a denoising step, the model performs bidirectional computation over the entire latent sequence, and an arbitrary latent-token prefix does not represent semantically complete progress along the diffusion trajectory. Although a DiT implementation may internally employ sequence parallelism or model-specific partitioning, such boundaries are implementation artifacts rather than portable scheduling points that can be safely exposed to a serving runtime.

Instead, DiT requests expose natural scheduling opportunities at stage and denoising-step boundaries. A request first executes conditioning encoders (e.g., text or image encoders) to produce embeddings and initialize latent states. The denoising stage then repeatedly applies the DiT model across a sequence of timesteps, progressively advancing the diffusion trajectory. Finally, a VAE [15] decoder converts the denoised latent into the final image or video.

As illustrated in Fig. 2, completing an encoder stage, a decoder stage, or a denoising step produces a semantically

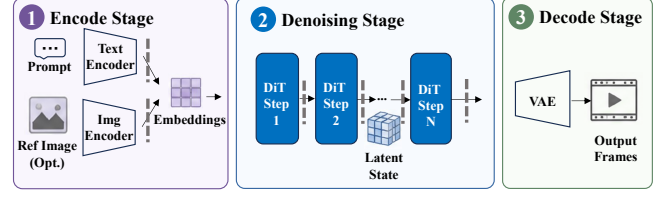


Figure 2. Structure of a diffusion serving request. Encoding produces conditioning embeddings, denoising iteratively advances the diffusion trajectory, and VAE decoding generates the final output. Trajectory task boundaries provide semantically valid rescheduling points for runtime adaptation.

complete execution state that can be safely transferred, resumed, or executed under a different resource configuration. Each such point is a *rescheduling boundary* at which the runtime may reconsider task placement and GPU allocation. GF-DiT represents each stage or denoising step between these boundaries as an independently schedulable *trajectory task*. By exposing these boundaries throughout request execution, GF-DiT creates opportunities for online parallelism adaptation while preserving request semantics.

2.2 Predictable Execution Structure

A key property of DiT workloads is that much of a request’s execution structure is known before execution begins. Request parameters or service defaults specify the output resolution, video duration, and denoising-step count at admission time. For a fixed model and pipeline, these parameters determine the latent sequence length and the overall diffusion trajectory.

Once a request is decomposed into trajectory tasks, its execution graph becomes largely deterministic. The number of tasks, their dependency structure, and the remaining execution path are all known a priori. Moreover, each trajectory task corresponds to a known model stage or denoising step operating on a known input shape. Its execution cost under a candidate parallel configuration can therefore be estimated from profiling data rather than discovered only after execution completes.

This predictability fundamentally distinguishes DiT serving from many dynamic serving workloads. Because future execution costs remain largely predictable, the runtime can reason about alternative resource allocations before they are applied. Combined with the rescheduling opportunities provided by trajectory task boundaries, this property makes online parallelism adaptation both practical and effective.

2.3 Why Parallelism Should Be Schedulable

We next use microbenchmarks and trace-driven replay to show why fixed parallelism is fundamentally insufficient for DiT serving. Our results demonstrate that the optimal

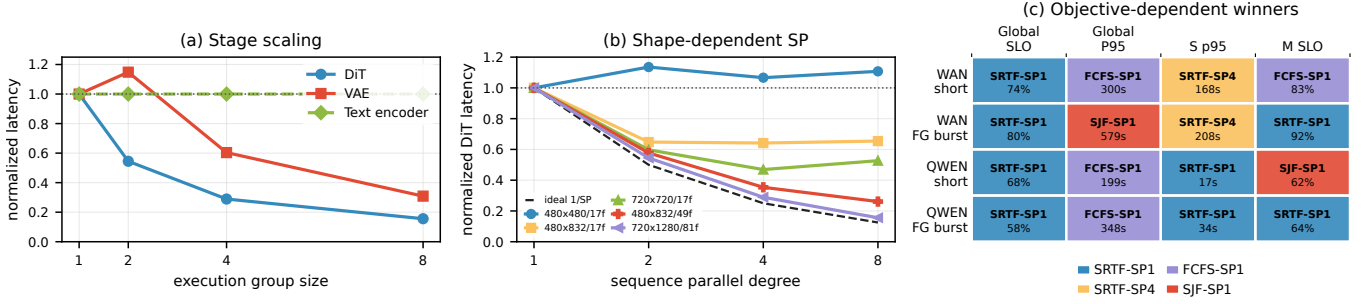


Figure 3. Motivating measurements for elastic DiT serving. (a) Different stages exhibit distinct scaling behavior and resource preferences. (b) The performance benefit of parallelism depends on request shape. (c) Different workload conditions favor different parallelism choices, indicating that no single parallel configuration is universally optimal.

degree of parallelism depends on the execution stage, request characteristics, and system conditions.

Stage-level heterogeneity. Fig. 3(a) reports representative stage latencies under different execution-group sizes. Different stages exhibit fundamentally different scaling behavior. Denoising tasks benefit from larger sequence-parallel groups because they dominate computation and operate on long latent sequences. In contrast, text encoding remains effectively single-rank, while VAE decoding exhibits a distinct scaling profile. Consequently, a single request-wide parallel configuration either wastes resources on lightweight stages or under-provisions compute-intensive denoising tasks.

Shape-dependent parallelism. Fig. 3(b) reports the latency of a denoising task, normalized to single-GPU execution, across request shapes and sequence-parallel degrees. Larger image and video requests expose more computation per denoising step and therefore benefit more from additional GPUs. Smaller requests, however, often fail to amortize communication and synchronization overheads, resulting in limited gains from larger execution groups. Even for the same model and execution stage, the optimal parallel configuration depends strongly on request shape.

System-dependent parallelism. Fig. 3(c) evaluates different serving configurations using trace-driven replay. Under lightly loaded conditions, allocating more GPUs to individual requests may minimize latency. Under heavier workloads, however, reducing per-request parallelism often improves overall throughput and SLO attainment by increasing concurrency. As workload composition, request arrivals, and service objectives evolve over time, the preferred parallel configuration changes accordingly.

Together, these observations show that no single parallel configuration is optimal throughout a request’s lifetime. Different execution stages, request characteristics, and system states prefer different degrees of parallelism. This motivates the central design principle of GF-DiT: rather than treating parallelism as a fixed execution configuration chosen at admission time, a DiT serving system should treat *GPU parallelism as a first-class schedulable resource*. Trajectory task

boundaries create opportunities for adaptation, predictable execution structure makes adaptation feasible, and workload heterogeneity makes adaptation necessary. Enabling such adaptation efficiently motivates the asynchronous execution abstraction, group-free collectives, and policy-programmable runtime presented in the remainder of the paper.

3 GF-DiT Abstraction and Overview

GF-DiT turns parallelism from a static deployment choice into a schedulable resource. It exposes runtime-visible trajectory tasks along each diffusion trajectory, lets a policy bind each ready task to an execution layout, and realizes the resulting decisions through an asynchronous runtime. This section defines the abstraction and shows how its components fit together; Sections 4 and 5 describe the communication mechanism and runtime implementation.

3.1 Reschedulable Trajectory Tasks

GF-DiT represents each diffusion request as a placement-agnostic *trajectory task graph*. Its nodes are independently schedulable trajectory tasks, and its edges are artifact dependencies. A trajectory task may represent a model stage, such as encoding, latent preparation, or decoding, or a single denoising step. Completing a task produces a well-defined model state, so the runtime may reconsider task order, placement, and parallelism at the next boundary without exposing partially updated model internals.

Trajectory tasks communicate through *logical artifacts*. An artifact may represent conditioning features, latent state, scheduler metadata, decoded outputs, or other model-specific state needed by downstream tasks. At the abstraction level, an artifact records a dependency and semantic role but not a physical layout. The same artifact may later be materialized as replicated tensors, sequence-parallel shards, or model-specific metadata according to the layouts selected for its producer and consumer.

This graph exposes two properties of diffusion execution to the runtime. First, task boundaries are explicit rescheduling points at which GPU allocation can change. Second, the

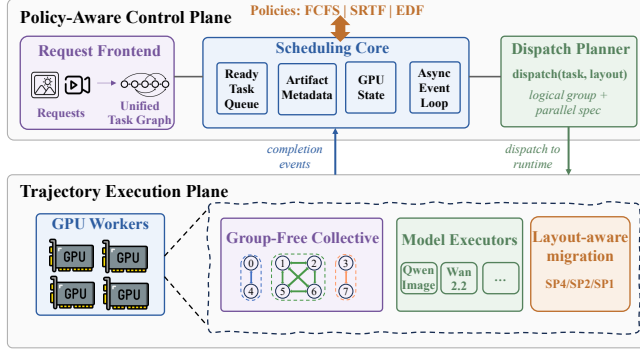


Figure 4. GF-DiT system overview. Incoming requests are converted into trajectory task graphs, scheduled by the control plane through a programmable policy interface, and dispatched to workers under dynamic execution layouts. Group-free collectives and layout-aware artifact migration make these dynamic layout choices executable at runtime.

graph and request shape are largely known at admission, allowing the runtime to estimate remaining work and compare candidate parallel configurations before a task executes.

3.2 Execution Layouts and Policy Interface

A GF-DiT policy schedules a ready trajectory task by assigning it an *execution layout*. An execution layout consists of an ordered logical execution group and a parallel specification describing how the task uses that group. For example, a DiT task may use a multi-rank sequence-parallel layout, whereas a lightweight encoder task may use a single rank.

At each scheduling point, the policy observes ready tasks, request metadata, resource availability, optional deadlines or priorities, and cost estimates for candidate layouts. It returns dispatch decisions of the form

$$(task, executionlayout).$$

This interface lets policies jointly control task order, placement, and parallelism. A policy may prioritize short requests, allocate additional ranks to an urgent request, or shrink a running request at its next trajectory boundary to admit new work. The policy operates only on logical tasks, artifacts, and execution groups. It does not construct communicators, invoke model stages, or plan tensor transfers. The runtime validates each decision and handles these execution mechanics. In particular, group-free collectives make policy-selected logical groups executable without serving-path communicator construction, while layout-aware artifact migration reconstructs artifacts when consecutive tasks use different layouts.

3.3 System Overview

Fig. 4 shows how the abstraction maps to the system. A model-specific adapter converts each incoming request into

Table 1. NCCL subgroup setup costs on an 8-GPU server.

Group size	new_group (ms)	First coll. (ms)	Warm coll. (ms)	Memory (MB/GPU)
2	0.77	217.3	0.19	484
4	0.46	532.0	0.56	535
6	0.57	663.5	0.72	535
8	0.57	777.7	0.89	535

trajectory tasks and logical artifacts. The control plane maintains graph dependencies, artifact metadata, resource state, and policy-visible request information. When tasks become ready or resources are released, it invokes the policy and dispatches selected tasks under their assigned execution layouts. The execution plane consists of GPU workers, model executors, and communication runtimes. Workers execute tasks asynchronously and report progress to the control plane, allowing scheduling and CPU-side preparation to overlap with GPU execution. Group-free collectives support dynamic execution groups, and layout-aware artifact migration reconstructs intermediate artifacts when a policy changes placement or parallelism across task boundaries.

The same control-plane abstraction also supports offline policy exploration. The online runtime completes dispatched tasks through GPU workers; the simulator instead produces completion events from profiled task costs. Because both backends preserve the same task graph, resource state, and policy interface, policies evaluated in simulation can be deployed without rewriting their decision logic.

4 Group-Free Collectives

The abstraction in Section 3 lets policies assign trajectory tasks to dynamic execution layouts. DiT tasks often need subgroup collectives within those layouts, such as all-gather or all-to-all operations used by sequence-parallel execution [45]. GF-DiT executes these operations with group-free collectives (GFC): the runtime pays one world-level communication setup cost at initialization and represents each dynamic subgroup as lightweight metadata.

4.1 Limits of Process-Group-Based Collectives

Conventional collective libraries expose subgroup communication through communicators or process groups. A process group fixes the participating ranks and provides the state needed by collective operations issued within that group. This model works well for static model-parallel deployments, where the same tensor-parallel or sequence-parallel groups are reused for the lifetime of the server.

Policy-programmable diffusion serving changes this assumption. A throughput policy may run many requests on small groups to increase concurrency, while a deadline-aware policy may temporarily allocate a larger group to an urgent request. A single request may also use different groups for different points on its trajectory: encoder stages may run

on one rank, early DiT tasks may run on a larger sequence-parallel group, and later tasks may shrink or move as load changes. These decisions create subgroup rank sets that are chosen online rather than fixed at server startup.

Creating a process group for each selected rank set is a poor fit for the serving path. Construction involves host-side coordination and communicator initialization, so it adds latency exactly when the scheduler is making a fine-grained placement decision. Pre-creating candidate groups removes serving-path setup, but the number of subgroups grows exponentially with rank count; even a restricted set consumes memory and limits the policy to choices known ahead of time. For example, pre-creating every size-2 and size-4 group already requires $\binom{8}{2} + \binom{8}{4} = 98$ process groups on an 8-GPU server. On an NVL72 domain, the same restricted set grows to $\binom{72}{2} + \binom{72}{4} = 1,031,346$ groups. Given the nontrivial per-group memory footprint measured below, exhaustive pre-creation is impractical even before considering other group sizes.

Table 1 quantifies this cost on an 8-GPU server. The explicit `new_group` call returns in sub-millisecond time, but the first collective on the subgroup triggers hundreds of milliseconds of cold-path initialization and reserves roughly 0.5 GB of GPU memory. Warm collectives are fast; the problem is the one-time setup and memory footprint that dynamic policies would pay for each newly selected rank set.

GF-DiT therefore separates execution groups from communicators. A policy chooses a logical execution group as part of a task’s layout, while the communication runtime executes collectives for that group using GFC descriptors and world-level communication state.

4.2 Scope and Ordering Assumptions

GFC targets a controlled serving setting rather than arbitrary user-issued collective communication. The control plane decides which trajectory tasks run, which ranks participate in each layout, and which subgroup collectives those tasks issue. Executors invoke collectives through the runtime interface instead of directly constructing communicators, allowing GF-DiT to support dynamic subgroups without exposing a general distributed communicator API.

The main correctness requirement is ordering. A rank may participate in many logical groups over time, and these groups may overlap. For example, ranks 0 and 1 may first communicate as part of group $\{0, 1, 2, 3\}$ and later as part of group $\{0, 1\}$. If rank 0 enters these shared collective instances in a different order from rank 1, then a synchronization signal from one instance may be mistaken for another. This can lead to incorrect data movement or deadlock [25].

GF-DiT therefore assumes *pairwise-consistent ordering*: for any pair of ranks, the collective instances in which both ranks participate are issued in the same order at both ranks. This assumption is weaker than requiring a single global order over all collectives. Two disjoint groups may proceed

independently, and groups that overlap only partially need only agree on the relative order observed by each shared rank pair. Under this runtime model, GF-DiT enforces pairwise-consistent ordering through centralized scheduling and ordered submission. The scheduler is the only component that creates dynamic execution layouts, and each worker submits collectives for assigned tasks to a local ordered communication stream. Thus, any two ranks that appear together in multiple groups observe the same order of their shared collective instances.

4.3 Symmetric Buffers and Logical Groups

Group-free collectives separate communication state from subgroup membership. At initialization, GF-DiT performs a single world-level setup in which all ranks allocate and register symmetric communication buffers [29]. These buffers provide a common addressable substrate for data movement and synchronization across ranks. This setup is paid once when the server starts, rather than each time a policy creates a new execution group.

At runtime, a subgroup is represented by a logical group descriptor containing the ordered participating ranks, a runtime group identifier, and the local rank’s position in that order. Creating a logical group is therefore a metadata operation: the runtime constructs no communicator, allocates no heavy per-group communication state, and does not require non-members to join. A trajectory task can use a logical group once the participating ranks receive the same descriptor from the control plane. In our implementation, registration takes approximately 60 μ s: CPU-side descriptor book-keeping plus one host-to-device copy of the participating rank IDs, with no communicator initialization or subgroup warmup.

The descriptor gives each rank enough information to interpret collective semantics inside the group. For example, it defines the local rank index used by a sequence-parallel DiT executor and maps group-local peers to global ranks for communication. The symmetric buffers provide the storage and addressing needed to exchange data with those peers. Together, the descriptor and the world-level buffers replace the role that a per-subgroup process group would normally play in dynamic execution.

GF-DiT keeps this interface separate from model tensors. Executors operate on ordinary input and output tensors; when a collective is issued, the communication runtime stages data into a symmetric buffer region, performs subgroup data movement, and copies results back if needed. This staging hides GFC from model code while allowing the runtime to choose different backends for the same logical group abstraction.

4.4 Edge-Based Agreement for Overlapping Groups

Logical group descriptors and symmetric buffers make subgroup communication addressable, but they do not establish

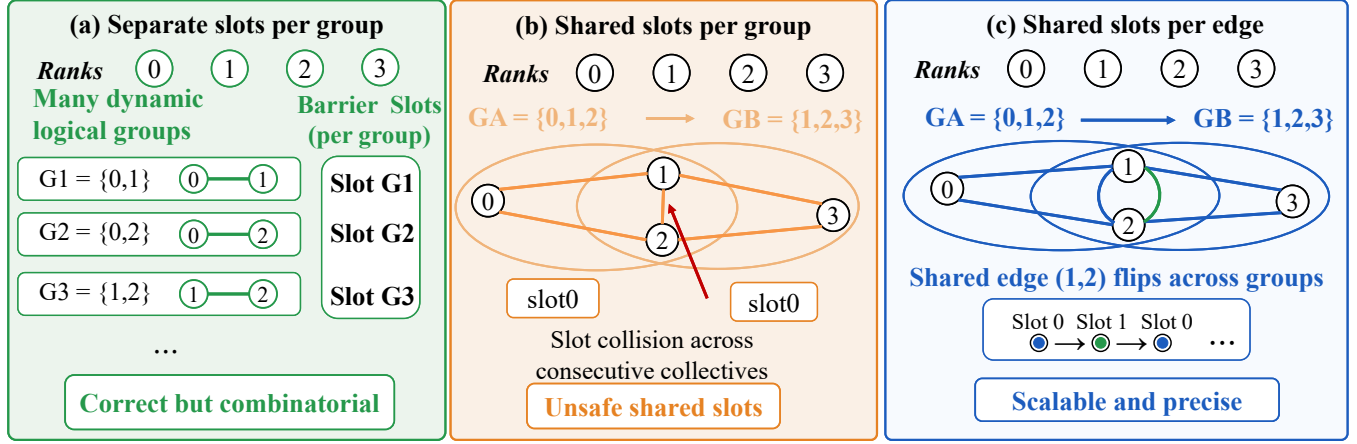


Figure 5. Signal-state designs for dynamic overlapping groups. (a) Separate slots for every logical group avoid collisions but require combinatorial per-group state. (b) Sharing slots across groups allows consecutive overlapping collectives to collide. (c) GF-DiT assigns double-buffered phase state to each rank edge, so a shared edge flips slots consistently across groups without allocating per-group signal state.

agreement on collective instances. Since logical groups can overlap, the same pair of ranks may communicate together in different groups over time. The runtime must ensure that a signal written by one rank for one collective is interpreted by its peer as belonging to that same collective.

GF-DiT identifies each collective instance with a synchronization token derived from the runtime session, the logical group identity, and a per-group logical epoch. The epoch advances whenever the group issues a new collective. The token therefore tells a peer which collective instance a signal belongs to, even if the same signal slot is reused later.

The remaining problem is deciding when a signal slot can be reused. A single slot per sender-receiver pair is unsafe: a rank could publish the next token before its peer has observed the previous one, overwriting the only evidence of arrival. Double buffering fixes this local lifetime problem by alternating between two slots, so instance $N + 1$ does not overwrite the token for instance N . However, the phase bit that selects the slot must advance in the same way at both endpoints for exactly the collective instances they share.

Two group-level slot designs are unsatisfactory. Giving every possible logical group its own barrier slots is correct, but the number of possible groups is combinatorial. Reusing one shared slot pool across logical groups is compact but unsafe: consecutive overlapping groups can make their shared rank pair publish to the same slot before the earlier token has been consumed, as in Fig. 5(b). The slot sequence must therefore be owned by the entity that actually observes reuse, namely the rank pair.

GF-DiT attaches the phase bit to each ordered rank edge. A group barrier requires each rank to agree with every other rank in the logical group and completes only after all pairwise edges observe the expected token. Fig. 5 contrasts this

Algorithm 1: Per-edge flip agreement for a GFC collective

Input: Logical group G , local rank $r \in G$, collective instance c

- 1 **State:** each local edge (r, p) keeps a one-bit phase $\phi_{r,p}$ and two signal slots;
 - 2 **Invariant:** for every rank pair, shared collective instances are submitted in the same order at both endpoints;
 - 3 $\tau \leftarrow \text{TOKEN}(G, c)$;
 - 4 **foreach** $p \in G \setminus \{r\}$ **do**
 - 5 $e \leftarrow (r, p)$;
 - 6 $s[p] \leftarrow \phi_{r,p}$;
 - 7 $\phi_{r,p} \leftarrow 1 - \phi_{r,p}$; // Flip phase
 - 8 $\text{PUBLISH}(e, s[p], \tau)$;
 - 9 **foreach** $p \in G \setminus \{r\}$ **do**
 - 10 $e' \leftarrow (p, r)$;
 - 11 wait until $\text{OBSERVED}(e', s[p], \tau)$;
-

design with group-level alternatives. Because slot ownership follows rank pairs, unrelated groups cannot overwrite one another, and overlapping groups interact only through the edges whose endpoints actually need to agree.

Algorithm 1 intentionally hides the signal layout. In the implementation, each ordered rank pair owns two signal slots. A rank publishes to the peer’s incoming slot and waits on the reciprocal slot. The slot is selected by the parity of a local per-edge sequence counter. Because the control plane submits pair-sharing collectives in the same order at both endpoints, the two ranks select matching slots for the same collective instance without exchanging slot state.

The protocol requires no separate acknowledgement phase. Because each rank submits edge instances on one ordered communication stream, a slot used by edge instance N is not reused until instance $N+2$. Before a rank can publish $N+2$, its local $N+1$ instance must have returned; that return implies that the peer published $N+1$, which occurs only after the peer consumed the token for N . The double buffer is therefore sufficient to prevent a token from being overwritten before its peer observes it. Publish and observe use system-scope release and acquire operations, respectively, so data movement begins only after all pairwise arrivals are visible.

This design relies on the pairwise-consistent ordering assumption from Section 4.2. Under that assumption, the next signal on an edge corresponds to the same collective instance at both endpoints, while the token detects stale or mismatched observations. GF-DiT satisfies the assumption because the centralized control plane creates dynamic groups and workers submit collectives through ordered per-rank communication streams.

4.5 Backend-Aware Collective Execution

After a logical group has been registered and its collective instance reaches agreement, GF-DiT executes data movement with a backend chosen for the message size and communication pattern. The GFC interface is backend independent: executors issue subgroup collectives over ordinary tensors, while the communication runtime maps each operation to CUDA-kernel communication, copy-engine transfers, or TMA-style transfers when available. This separation keeps model code independent of the communication mechanism.

A practical challenge is that executor tensors are not required to reside in symmetric communication buffers. Requiring all model intermediates to be allocated from a special symmetric-memory pool would make integration intrusive and would complicate existing model executors. GF-DiT therefore uses staging buffers. For each collective, the runtime copies the local input slice from the executor tensor into a symmetric buffer region, performs remote data movement through the selected backend, and writes the result back to the executor-visible output tensor.

Staging adds local copy work, but it also creates an opportunity for pipelining. GF-DiT divides the payload into communication chunks so that local staging and remote data movement can be overlapped. While one chunk is being transferred through the selected backend, the runtime can stage another chunk between executor tensors and symmetric buffers. This hides part of the local-copy overhead without requiring executors to allocate their tensors from a symmetric-memory pool.

Different backends are preferable at different message sizes. CUDA-kernel communication has low setup overhead and can combine data movement with simple packing or reduction logic; copy-engine transfers reduce SM interference

for larger point-to-point payloads; and TMA-style transfers, when available, improve bulk movement for structured tensor tiles. GF-DiT uses a selector populated from microbenchmark results to choose a backend and chunk size for each collective pattern and message-size range.

This backend selection is hidden behind the group-free collective API. The scheduler chooses a trajectory task’s execution layout, and the executor issues the collective required by its parallel specification. The communication runtime then handles agreement, staging, backend selection, and data movement. As a result, dynamic execution groups remain policy-level objects, while the low-level communication path can adapt to hardware and message size.

5 GF-DiT Implementation

We implement GF-DiT in vLLM-Omni as an event-driven control plane and a set of execution-plane components.

5.1 Event-Driven Control and Execution Planes

GF-DiT separates scheduling state from task execution. The control plane owns request admission, trajectory task graphs, dependency state, artifact metadata, resource availability, and policy invocation. The execution plane contains worker processes, model executors, and communication runtimes; workers execute assigned trajectory tasks and report events back to the control plane.

The key control-path event is dispatch completion. When the control plane dispatches a task, the runtime resolves its logical execution group, prepares required inputs, allocates output artifact handles, records their expected layouts, and sends the task to participating workers. Once this CPU-side dispatch finishes, the control plane can prepare successor state, plan later migrations, and issue independent work while the GPU task is still running.

Device execution is tracked separately. Workers record CUDA events around task execution, and a monitor thread reports execution-start and execution-completion events to the control plane. These events mark device completion, materialize output artifacts, release worker resources, detect failures, and calibrate the runtime cost model with measured task durations. Consumers remain blocked until their inputs are materialized, but preparation of scheduling state does not wait for device completion. Separating dispatch completion from device completion keeps scheduling asynchronous with GPU execution. Policy evaluation, dependency updates, migration planning, and other CPU-side work overlap with device execution instead of extending the GPU critical path at every trajectory boundary.

5.2 Model Adapters

GF-DiT keeps model-specific logic behind a narrow adapter interface. A model adapter translates a diffusion pipeline into

three runtime components: a request converter, task executors, and artifact codecs. The converter maps an incoming request to trajectory tasks and logical artifacts; executors run assigned tasks under a chosen layout; and codecs describe logical artifact layouts under different parallel settings.

The request converter determines the task granularity exposed to scheduling. It emits stage tasks for pipeline components such as text encoding and VAE decoding, as well as one task for each denoising step. Each executor accepts a task description and execution layout, invokes the corresponding model stage, and uses the communication runtime when its parallel specification requires collectives.

Artifact codecs connect model-specific tensor structures to the placement-agnostic task graph. For each logical artifact, a codec identifies replicated, sharded, and metadata-only fields and reports the global shape and per-rank slice of tensor fields under a given layout. The runtime uses these views to determine whether an artifact can be consumed directly or requires migration. This interface keeps scheduling policies independent of model internals. A policy sees ready tasks, request metadata, resource availability, and cost estimates, but not how latents are sharded, scheduler state is serialized, or a model stage invokes collectives. Adding a new diffusion pipeline therefore requires implementing an adapter rather than rewriting policies or embedding model-specific migration logic in the scheduler.

5.3 Layout-Aware Artifact Migration

When adjacent trajectory tasks use different execution layouts, the runtime must reconstruct their logical artifacts without exposing migration decisions to the scheduling policy. GF-DiT derives the required transfers from the producer and consumer artifact views in three steps: layout exchange, migration planning, and distributed execution.

First, the leader of the source group obtains source and destination views of each migrated artifact from the participating ranks. The adapter’s codec reports whether each field is replicated, sharded, or metadata-only; for tensor fields, it also reports the global shape and per-rank slice under each layout.

Second, the leader of the source group derives a migration plan from the two views. For a sharded tensor field, the planner intersects each source-owned slice with each destination-required slice. Every non-empty intersection becomes a transfer entry containing the source rank, destination rank, source tensor range, destination tensor range, and byte size.

Third, the leader of the source group distributes the plan to participating ranks. Each rank extracts its local actions, packs required tensor ranges, exchanges data with peers through point-to-point transfers, and installs received ranges into the destination artifact representation. The consumer can then read the artifact under its assigned layout.

GF-DiT executes migration transfers through the same group-free communication substrate, avoiding PyTorch [4] point-to-point paths that may silently construct two-rank communication groups in the background. For each migration edge, the runtime uses a logical pair group and the agreement mechanism from Section 4; larger migrations are decomposed into multiple transfer entries.

5.4 Scheduling Policies

GF-DiT exposes scheduling as a policy-layer decision. A policy observes ready trajectory tasks, request metadata, resource availability, and cost estimates, then returns a task ordering and execution layout for each dispatched task. We implement three representative policies.

FCFS with workload-aware group assignment. This policy partitions the cluster into worker groups and serves requests in FCFS order. When a task becomes ready, it is assigned to the feasible group with the lowest estimated queued workload. This approximates a throughput-oriented configuration: simple request ordering with runtime-managed load balancing across independent groups.

SRTF with per-rank local queues. This policy first assigns each request to a feasible rank using estimated queued workload. Each rank then schedules its own ready trajectory tasks by shortest remaining trajectory work [30], estimated from request shape, trajectory position, and profiled task costs. Compared with FCFS, SRTF uses diffusion’s predictable costs to reduce mean latency, while keeping the ordering decision local to the rank that owns the request.

EDF with best-fit parallelism. This policy targets SLO settings. It orders ready tasks by earliest deadline first [18], evaluates candidate layouts with the cost model, and selects the smallest parallel configuration predicted to meet the deadline. If a request is at risk of missing its deadline, the policy can assign a larger group at the next trajectory boundary, demonstrating SLO-aware dynamic placement without hard-coding deadline logic into model executors.

These policies are deliberately simple: they differ only in task ranking and layout choice. Dependency tracking, asynchronous dispatch, dynamic-group communication, and artifact migration are handled by the runtime.

5.5 Simulator and Cost Model

The same abstraction supports offline policy exploration. As shown in Section 2.3, the preferred scheduling policy depends on the workload mix, request shapes, serving objective, and available ranks. A deployment may therefore need to compare several policies before choosing one for the online serving path.

As described in Section 2.2, diffusion requests expose predictable trajectory task graphs and layout-specific costs. The simulator replaces worker execution with cost-model completion events while preserving task readiness, dependency updates, resource allocation, and policy invocation. Its cost

model is populated from profiled task latencies indexed by model, task type, request shape, and parallel configuration.

Because the simulator and online runtime share the same policy interface, a policy selected offline can be deployed without rewriting its decision logic. In simulation, dispatch decisions produce cost-model completion events; in the online runtime, the same decisions are executed by GPU workers and reported as real completion events. The simulator is therefore an alternative execution backend for the same trajectory abstraction rather than a separate scheduling model.

6 Evaluation

We evaluate GF-DiT on image and video diffusion serving workloads.

6.1 Experimental Setup

We implement GF-DiT on vLLM-Omni 0.17.0 [45] using Python 3.12, CUDA 12.9, PyTorch 2.10 [4], Triton 3.6.0 [37], and NCCL 2.27.5 [25]. Experiments run on a 4-GPU H20 server and an 8-GPU A100 server, using Wan2.2-5B [39] for video generation and Qwen-Image [41] for image generation.

For each model and platform, we generate fixed-duration request traces with arrival rates calibrated from measured platform throughput, so policies are compared under comparable serving pressure rather than a single absolute request rate. Fig. 7 shows two workloads: a compact mixed-arrival period and a foreground-burst setting where bursts of short requests arrive while longer requests may already be in flight. Together, they test whether the runtime can preserve mixed-request concurrency while giving urgent short requests priority in scheduling. For Wan2.2, short, medium, and long (S/M/L) requests generate 480×832 videos with 49 frames, 480×832 videos with 81 frames, and 720×1280 videos with 81 frames, respectively. For Qwen-Image, S/M/L requests generate 512×512, 1024×1024, and 1536×1536 images.

Each request’s SLO is defined relative to its profiled standalone service time on the same model and platform. A class- c request arriving at time a receives a scheduler-visible deadline $a + \alpha_c T_c$, where T_c is the profiled service time of its request shape. We use $\alpha_{S/M/L} = 2.0/2.5/3.5$ for Wan2.2 and $1.5/2.0/6.0$ for Qwen-Image, giving short interactive requests tighter relative deadlines while leaving longer requests more queueing slack. The multipliers are fixed across platforms and workloads, while T_c is re-profiled per platform. We choose these representative multipliers before comparing policies and do not tune them per policy. The client-visible deadline additionally includes a fixed allowance for non-profiled overheads: 5 s for Wan2.2 and 1 s for Qwen-Image. We report SLO attainment as the fraction of submitted requests that complete by this deadline.

Each request also has a loose client-side timeout, 1500 s for image generation and 3600 s for video generation. Timed-out requests are recorded as failures and SLO violations. Latency

statistics include only completed requests, and throughput is measured as completed requests per second, so latency results for policies with failures are optimistic.

6.2 Baselines

We compare against the native vLLM-Omni serving path, denoted Legacy, and the policies from Section 5.4. Legacy uses vLLM-Omni’s original fixed-pipeline execution with static parallelism over the full machine. EDF, SRTF-SP1, SRTF-SPmax, and FCFS-SP1 are implemented on top of the GF-DiT policy interface.

6.3 Main Results

Fig. 6 shows that different objectives favor different policies. EDF achieves the highest SLO attainment in every platform-model-workload setting, with large gains in bursty or highly concurrent cases: on A100 Wan2.2 foreground burst, SLO attainment improves from 7.5% with Legacy to 88.8% with EDF; on H20 Qwen-Image foreground burst, it improves from 6.2% to 72.8%. Across all main-result settings, GF-DiT improves throughput by up to 6.01×, reduces mean latency by up to 95.3%, and reduces SLO violation rate by up to 89.6% over Legacy. EDF improves deadline attainment because it orders trajectory tasks by deadline and chooses parallelism from estimated completion time instead of using one fixed execution order and static parallel configuration.

The latency metrics explain why no single policy dominates all objectives. Legacy and SRTF-SPmax can reduce the service time of individual large tasks with large execution groups, but they reduce concurrency and increase queueing under mixed workloads. Smaller single-rank policies can therefore win on latency: SRTF-SP1 gives the lowest mean latency on several Qwen-Image settings, while FCFS-SP1 gives the lowest P95 latency on the A100 Qwen-Image short workload. These results support the main design goal of GF-DiT: scheduling strategy should be a policy-layer choice over one runtime substrate.

The loss of concurrency also causes a capacity collapse for the large-group policies on A100 Qwen-Image. On the foreground-burst workload, Legacy and SRTF-SPmax complete only 37% and 39% of submitted requests, respectively, while the other policies complete nearly all requests. These failures occur when requests remain in the serialized SP8 queue beyond the client timeout. Because failed requests are excluded from latency statistics, the reported latencies for these baselines are optimistic.

6.4 Runtime Overhead

Fig. 8 compares the native Legacy path, which uses fixed-pipeline execution with static parallelism, with GF-DiT pinned to FCFS-SP4, which uses the same FIFO order and a full-machine SP4 group. FCFS-SP4 closely matches Legacy

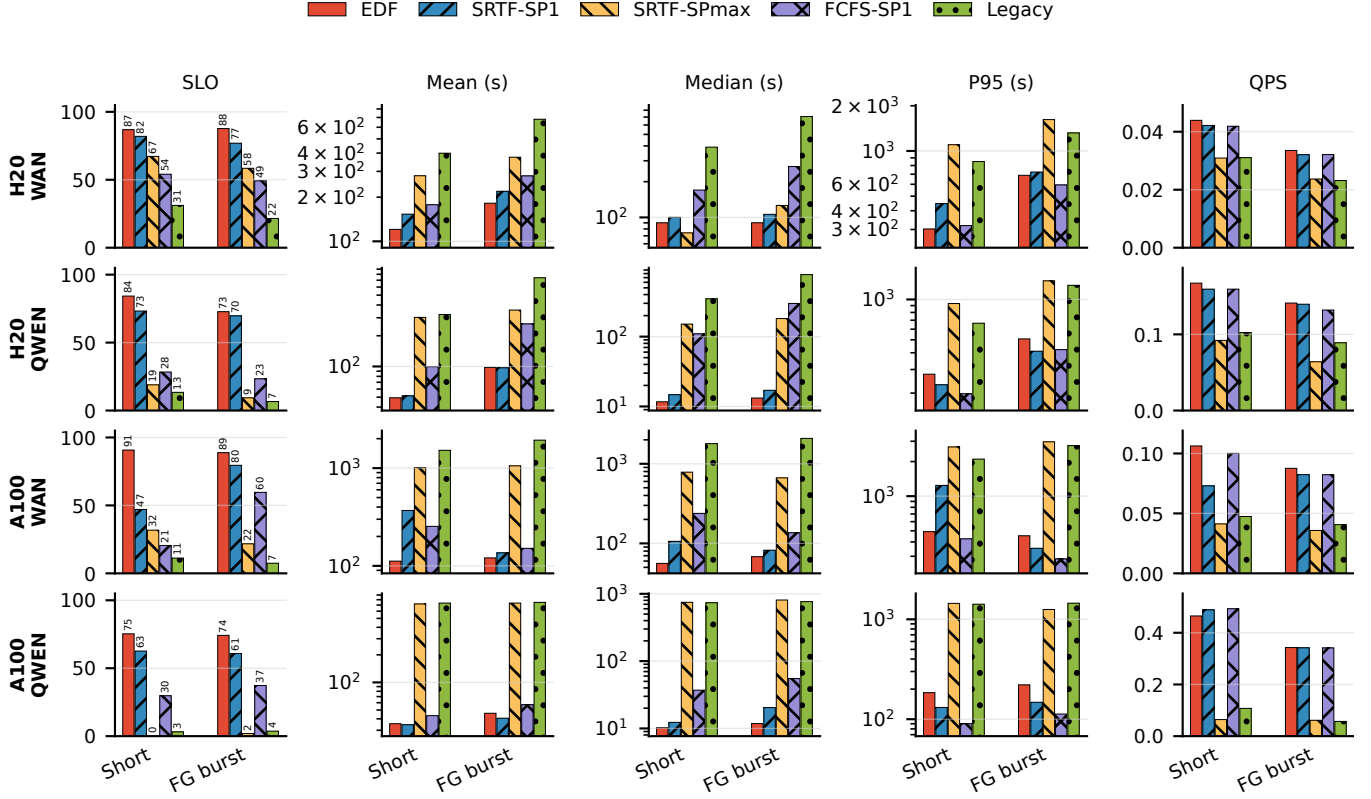


Figure 6. End-to-end serving results on H20 and A100. Each row is a platform-model pair, and each column reports one metric across the short and foreground-burst workloads. Legacy is the native vLLM-Omni fixed-pipeline execution path with static parallelism; the other policies are implemented on top of GF-DiT. SLO attainment includes failed requests as violations, while latency statistics are computed over completed requests only.

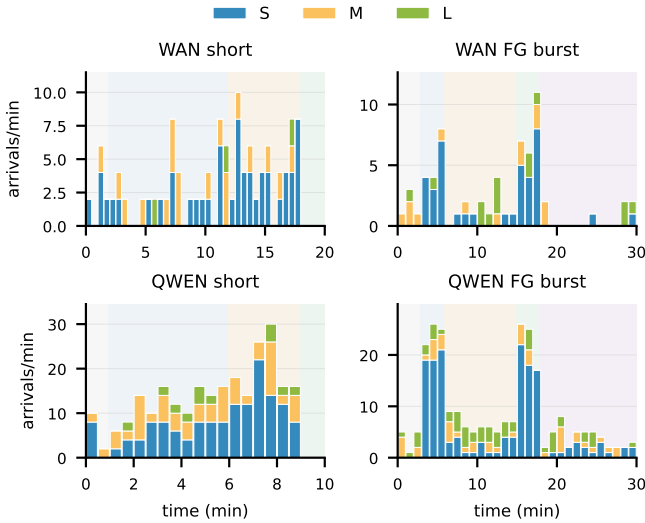


Figure 7. Arrival-rate timelines of the short and foreground-burst workloads used in the main results. Colors denote short (S), medium (M), and long (L) request classes.

in throughput and mean latency, showing that policy programmability introduces negligible overhead when the execution order and layout are fixed.

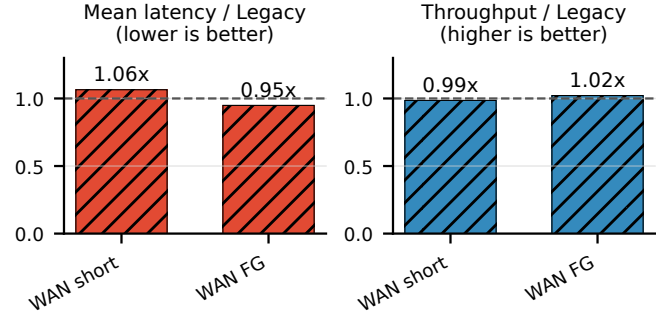


Figure 8. Runtime overhead relative to the native Legacy path on 4-GPU H20. FCFS-SP4 pins GF-DiT to the same FIFO order and static SP4 layout.

6.5 Group-Free Collective Performance

Fig. 9 compares steady-state GFC latency against NCCL with pre-initialized process groups. For small messages, GFC can be slower because its fixed synchronization and staging overheads are not yet optimized for this regime. Diffusion-serving collectives typically use larger intermediate tensors, for which these costs are amortized; in this range, GFC generally matches or outperforms NCCL. For example, GFC

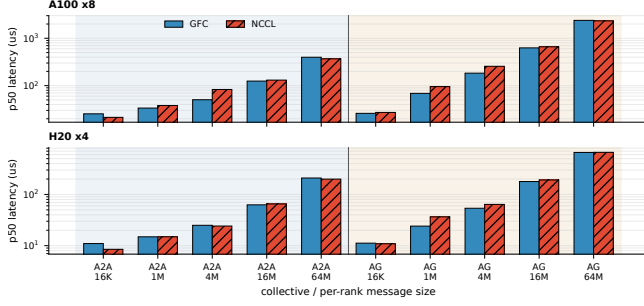


Figure 9. Latency of GFC and NCCL collectives on A100 and H20. We measure BF16 all-to-all (A2A) and all-gather (AG) across different per-rank message sizes.

reduces A100 4 MB all-to-all latency from 83.0 μ s to 50.4 μ s and H20 1 MB all-gather latency from 36.5 μ s to 24.1 μ s. Thus, GFC avoids serving-path process-group construction while maintaining competitive steady-state performance.

6.6 Scaling with Arrival Rate

Fig. 10 evaluates EDF and SRTF-SP1 as arrival rate increases. At low and moderate loads, EDF achieves higher SLO attainment by prioritizing earlier deadlines and adding parallelism when needed. Under sustained overload, however, EDF may keep rescuing tight-deadline requests with larger sequence-parallel groups, consuming more GPUs per request and delaying new arrivals. SRTF-SP1 then achieves higher SLO attainment by preserving single-rank concurrency and completing more requests with little remaining work. The crossover shows that deadline-aware parallelism helps when capacity is available but can reduce overall SLO attainment under overload.

6.7 Simulator Fidelity

Fig. 11 compares simulator predictions with real serving runs on a representative H20 workload. We compare the four programmable policies from the main results and include FCFS-SP4 as a static large-group baseline. Simulator predictions closely match real serving results: all five policies differ by at most 4.7 percentage points in overall SLO attainment. The simulator is therefore accurate enough to compare policy trends and identify candidates, while final selection still requires real-system validation.

7 Related Work

Diffusion model serving. Recent diffusion serving systems exploit stage- or step-level execution structure to improve serving efficiency. TridentServe dynamically allocates resources across encoding, denoising, and decoding

stages [42], while TetriServe and GenServe perform step-level scheduling for homogeneous and heterogeneous diffusion workloads [20, 44]. Other systems focus on batching, mixed-resolution serving, model cascades, or adapter-intensive workloads [3, 17, 22, 34, 35]. In contrast, GF-DiT introduces a programmable elastic runtime that treats GPU parallelism as a schedulable resource and supports dynamic parallelism adaptation under diverse serving objectives. Existing DiT inference systems provide efficient parallel execution layouts, including sequence, patch-pipeline, and CFG parallelism [8, 9, 21]. Frameworks such as vLLM-Omni, SGLang Diffusion, and FastVideo integrate these techniques into practical deployment stacks [10, 31, 45]. GF-DiT is complementary to these efforts: rather than proposing new parallel kernels, it enables existing execution layouts to be dynamically selected and reconfigured at runtime.

Collective communication and dynamic groups. Collective communication libraries such as NCCL, RCCL, UCC, and UCCL rely on statically defined communication groups [1, 25, 33, 38, 48], while recent work improves collective performance through synthesis, scheduling, and topology-aware optimization [6, 7, 14, 24, 26, 32, 40]. These systems optimize communication within a predefined group. In contrast, GF-DiT addresses the serving-runtime challenge of online execution-group reconfiguration through group-free collectives, eliminating expensive communicator construction on the serving path.

8 Conclusion

This paper presented **GF-DiT**, a policy-programmable runtime for elastic DiT serving. By treating GPU parallelism as a first-class schedulable resource, GF-DiT enables online adaptation of resource allocation to workload demands, system conditions, and service objectives. Through an asynchronous execution abstraction, group-free collectives, and a programmable runtime substrate, GF-DiT makes elastic parallelism practical for real-world DiT serving workloads, substantially improving throughput, latency, and SLO attainment over fixed-pipeline execution with static parallelism. More broadly, we believe that future generative AI serving systems should move beyond static resource allocation and embrace runtime-managed parallelism as a fundamental systems abstraction.

References

- [1] Advanced Micro Devices, Inc. 2026. ROCm Communication Collectives Library (RCCL). <https://rocm.docs.amd.com/projects/rccl/en/latest/>. Documentation.
- [2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming throughput-latency tradeoff in LLM inference with sarathi-serve. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation (Santa Clara, CA, USA) (OSDI'24)*. USENIX Association, USA, Article 7, 18 pages.

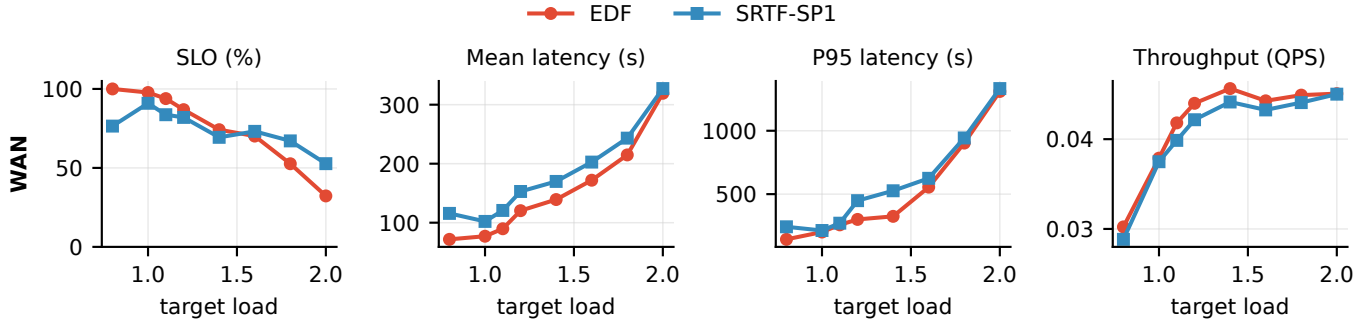


Figure 10. Performance of EDF and SRTF-SP1 under increasing arrival rates on the 4-GPU H2O Wan2.2 short workload. Target load is normalized to the estimated serving capacity.

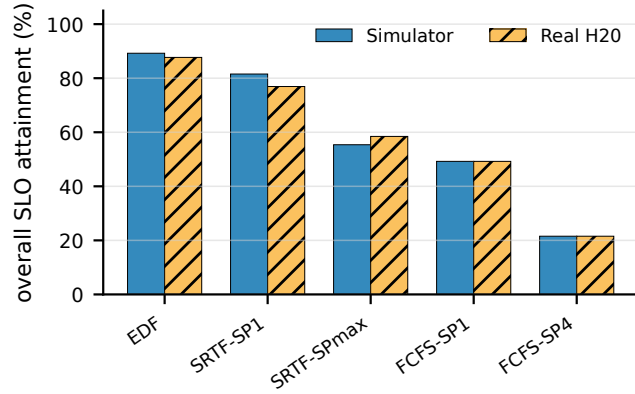


Figure 11. Simulator versus real 4-GPU H2O overall SLO attainment for the Wan2.2 foreground-burst workload. The simulator replays the exact request trace and policy logic using measured stage costs.

- [3] Sohaib Ahmad, Qizheng Yang, Haoliang Wang, Ramesh K. Sitaraman, and Hui Guan. 2025. DiffServe: Efficiently Serving Text-to-Image Diffusion Models with Query-Aware Model Scaling. arXiv:2411.15381 [cs.DC] <https://arxiv.org/abs/2411.15381>
- [4] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalam-barkar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, C. K. Luk, Bert Maher, Yunjie Pan, Christian Puhersch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Shunting Zhang, Michael Suo, Phil Tillet, Xu Zhao, Eikan Wang, Keren Zhou, Richard Zou, Xiaodong Wang, Ajit Mathews, William Wen, Gregory Chanan, Peng Wu, and Soumith Chintala. 2024. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (La Jolla, CA, USA) (ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 929–947. doi:10.1145/3620665.3640366
- [5] Tim Brooks, Bill Peebles, Connor Holmes, Will DePue, Yufei Guo, Li Jing, David Schnurr, Joe Taylor, Troy Luhman, Eric Luhman, Clarence Ng, Ricky Wang, and Aditya Ramesh. 2024. Video generation models as world simulators. (2024). <https://openai.com/research/video-generation-models-as-world-simulators>

- [6] Zixian Cai, Zhengyang Liu, Saeed Maleki, Madanlal Musuvathi, Todd Mytkowicz, Jacob Nelson, and Olli Saarikivi. 2021. Synthesizing optimal collective algorithms. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '21)*. ACM, 62–75. doi:10.1145/3437801.3441620
- [7] DeepSeek-AI. 2025. DeepEP: An Efficient Expert-Parallel Communication Library. <https://github.com/deepseek-ai/DeepEP>. Software library.
- [8] Jiarui Fang, Jinzhe Pan, Aoyu Li, Xibo Sun, and Jiannan Wang. 2025. PipeFusion: Patch-level Pipeline Parallelism for Diffusion Transformers Inference. arXiv:2405.14430 [cs.CV] <https://arxiv.org/abs/2405.14430>
- [9] Jiarui Fang, Jinzhe Pan, Xibo Sun, Aoyu Li, and Jiannan Wang. 2024. xDiT: an Inference Engine for Diffusion Transformers (DiTs) with Massive Parallelism. arXiv:2411.01738 [cs.DC] <https://arxiv.org/abs/2411.01738>
- [10] FastVideo Team. 2025. FastVideo: A Unified Inference and Post-Training Framework for Accelerated Video Generation. <https://github.com/hao-ai-lab/FastVideo>. Software.
- [11] Yu Gao, Haoyuan Guo, Tuyen Hoang, Weilin Huang, Lu Jiang, Fangyuan Kong, Huixia Li, Jiashi Li, Liang Li, Xiaojie Li, Xunsong Li, Yifu Li, Shanchuan Lin, Zhijie Lin, Jiawei Liu, Shu Liu, Xiaonan Nie, Zhiwu Qing, Yuxi Ren, Li Sun, Zhi Tian, Rui Wang, Sen Wang, Guoqiang Wei, Guohong Wu, Jie Wu, Ruiqi Xia, Fei Xiao, Xuefeng Xiao, Jiangqiao Yan, Ceyuan Yang, Jianchao Yang, Runkai Yang, Tao Yang, Yihang Yang, Zilyu Ye, Xuejiao Zeng, Yan Zeng, Heng Zhang, Yang Zhao, Xiaozheng Zheng, Peihao Zhu, Jiaxin Zou, and Feilong Zuo. 2025. Seedance 1.0: Exploring the Boundaries of Video Generation Models. arXiv:2506.09113 [cs.CV] <https://arxiv.org/abs/2506.09113>
- [12] Agrim Gupta, Lijun Yu, Kihyuk Sohn, Xiuye Gu, Meera Hahn, Li Fei-Fei, Irfan Essa, Lu Jiang, and José Lezama. 2023. Photorealistic Video Generation with Diffusion Models. arXiv:2312.06662 [cs.CV] <https://arxiv.org/abs/2312.06662>
- [13] Yoav HaCohen, Nisan Chiprut, Benny Brazowski, Daniel Shalem, Dudu Moshe, Eitan Richardson, Eran Levin, Guy Shiran, Nir Zabari, Ori Gordon, Poriya Panet, Sapir Weissbuch, Victor Kulikov, Yaki Bitterman, Zeev Melumian, and Ofir Bibi. 2024. LTX-Video: Realtime Video Latent Diffusion. arXiv:2501.00103 [cs.CV] <https://arxiv.org/abs/2501.00103>
- [14] Changho Hwang, Peng Cheng, Roshan Dathathri, Abhinav Jangda, Saeed Maleki, Madan Musuvathi, Olli Saarikivi, Aashaka Shah, Ziyue Yang, Binyang Li, Caio Rocha, Qinghua Zhou, Mahdieh Ghazimirsaeed, Sreevatsa Anantharamu, and Jithin Jose. 2026. MSCCL++: Rethinking GPU Communication Abstractions for AI Inference. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (USA) (ASPLOS '26)*. Association for Computing Machinery, New York, NY, USA, 1201–1215. doi:10.1145/3779212.3790188

- [15] Diederik P Kingma and Max Welling. 2022. Auto-Encoding Variational Bayes. arXiv:1312.6114 [stat.ML] <https://arxiv.org/abs/1312.6114>
- [16] Weijie Kong, Qi Tian, Zijian Zhang, Rox Min, Zuozhuo Dai, Jin Zhou, Jiangfeng Xiong, Xin Li, Bo Wu, Jianwei Zhang, Kathrina Wu, Qin Lin, Junkun Yuan, Yanxin Long, Aladdin Wang, Andong Wang, Changlin Li, Duojuan Huang, Fang Yang, Hao Tan, Hongmei Wang, Jacob Song, Jiawang Bai, Jianbing Wu, Jinbao Xue, Joey Wang, Kai Wang, Mengyang Liu, Pengyu Li, Shuai Li, Weiyan Wang, Wenqing Yu, Xincheng Deng, Yang Li, Yi Chen, Yutao Cui, Yuanbo Peng, Zhentao Yu, Zhiyu He, Zhiyong Xu, Zixiang Zhou, Zunnan Xu, Yangyu Tao, Qinglin Lu, Songtao Liu, Dax Zhou, Hongfa Wang, Yong Yang, Di Wang, Yuhong Liu, Jie Jiang, and Caesar Zhong. 2025. HunyuanVideo: A Systematic Framework For Large Video Generative Models. arXiv:2412.03603 [cs.CV] <https://arxiv.org/abs/2412.03603>
- [17] Suyi Li, Lingyun Yang, Xiaoxiao Jiang, Hanfeng Lu, Dakai An, Zhipeng Di, Weiye Lu, Jiawei Chen, Kan Liu, Yinghao Yu, Tao Lan, Guodong Yang, Lin Qu, Liping Zhang, and Wei Wang. 2025. KATZ: efficient workflow serving for diffusion models with many adapters. In *Proceedings of the 2025 USENIX Conference on Usenix Annual Technical Conference* (Boston, MA, USA) (USENIX ATC '25). USENIX Association, USA, Article 61, 16 pages.
- [18] C. L. Liu and James W. Layland. 1973. Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment. *J. ACM* 20, 1 (Jan. 1973), 46–61. doi:10.1145/321738.321743
- [19] Haoyu Lu, Guoxing Yang, Nanyi Fei, Yuqi Huo, Zhiwu Lu, Ping Luo, and Mingyu Ding. 2023. VDT: General-purpose Video Diffusion Transformers via Mask Modeling. arXiv:2305.13311 [cs.CV] <https://arxiv.org/abs/2305.13311>
- [20] Runyu Lu, Shiqi He, Wenxuan Tan, Shenggui Li, Ruofan Wu, Jeff J. Ma, Ang Chen, and Mosharaf Chowdhury. 2026. TetrisServe: Efficient DiT Serving for Heterogeneous Image Generation. arXiv:2510.01565 [cs.LG] <https://arxiv.org/abs/2510.01565>
- [21] Jiajun Luo, Yicheng Xiao, Jianru Xu, Yangxiu You, Rongwei Lu, Chen Tang, Jingyan Jiang, and Zhi Wang. 2025. Accelerating Parallel Diffusion Model Serving with Residual Compression. arXiv:2507.17511 [cs.CV] <https://arxiv.org/abs/2507.17511>
- [22] Michael Luo, Aaron Hao, Zhengxu Yan, Chengkun Cao, and Quang Luong Nhat Nguyen. 2026. DiT-Serve: An Efficient Serving Engine for Diffusion Transformers. <https://openreview.net/forum?id=NGNRc7rZBg>
- [23] Xin Ma, Yaohui Wang, Xinyuan Chen, Gengyun Jia, Ziwei Liu, Yuanfang Li, Cunjian Chen, and Yu Qiao. 2025. Latte: Latent Diffusion Transformer for Video Generation. arXiv:2401.03048 [cs.CV] <https://arxiv.org/abs/2401.03048>
- [24] Ziming Mao, Yihan Zhang, Chihan Cui, Zhen Huang, Kaichao You, Zhongjie Chen, Zhiying Xu, Zhenyu Gu, Scott Shenker, Costin Raiciu, Yang Zhou, and Ion Stoica. 2026. UCCL-EP: Portable Expert-Parallel Communication. arXiv:2512.19849 [cs.DC] <https://arxiv.org/abs/2512.19849>
- [25] NVIDIA Corporation. 2026. NVIDIA Collective Communication Library (NCCL). <https://developer.nvidia.com/nccl>. Software library.
- [26] Lichen Pan, Juncheng Liu, Yongquan Fu, Jinhui Yuan, Rongkai Zhang, Pengze Li, and Zhen Xiao. 2025. Comprehensive Deadlock Prevention for GPU Collective Communication. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*. ACM, 541–557. doi:10.1145/3689031.3717466
- [27] William Peebles and Saining Xie. 2023. Scalable Diffusion Models with Transformers. arXiv:2212.09748 [cs.CV] <https://arxiv.org/abs/2212.09748>
- [28] Adam Polyak, Amit Zohar, Andrew Brown, Andros Tjandra, Animesh Sinha, Ann Lee, Apoorv Vyas, Bowen Shi, Chih-Yao Ma, Ching-Yao Chuang, David Yan, Dhruv Choudhary, Dingkang Wang, Geet Sethi, Guan Pang, Haoyu Ma, Ishan Misra, Ji Hou, Jialiang Wang, Kiran Jagadeesh, Kunpeng Li, Luxin Zhang, Mannat Singh, Mary Williamson, Matt Le, Matthew Yu, Mitesh Kumar Singh, Peizhao Zhang, Peter Vajda, Quentin Duval, Rohit Girdhar, Roshan Sumbaly, Sai Saketh Rambhatla, Sam Tsai, Samaneh Azadi, Samyak Datta, Sanyuan Chen, Sean Bell, Sharadh Ramaswamy, Shelly Sheynin, Siddharth Bhattacharya, Simran Motwani, Tao Xu, Tianhe Li, Tingbo Hou, Wei-Ning Hsu, Xi Yin, Xiaoliang Dai, Yaniv Taigman, Yaqiao Luo, Yen-Cheng Liu, Yi-Chiao Wu, Yue Zhao, Yuval Kirstain, Zecheng He, Zijian He, Albert Pumarola, Ali Thabet, Arsiom Sanakoyeu, Arun Mallya, Baishan Guo, Boris Araya, Breena Kerr, Carleigh Wood, Ce Liu, Cen Peng, Dmitry Vengertsev, Edgar Schonfeld, Elliot Blanchard, Felix Juefei-Xu, Fraylie Nord, Jeff Liang, John Hoffman, Jonas Kohler, Kaolin Fire, Karthik Sivakumar, Lawrence Chen, Licheng Yu, Luya Gao, Markos Georgopoulos, Rashel Moritz, Sara K. Sampson, Shikai Li, Simone Parmeggiani, Steve Fine, Tara Fowler, Vladan Petrovic, and Yuming Du. 2025. Movie Gen: A Cast of Media Foundation Models. arXiv:2410.13720 [cs.CV] <https://arxiv.org/abs/2410.13720>
- [29] PyTorch Contributors. 2025. PyTorch Symmetric Memory. https://docs.pytorch.org/docs/stable/symmetric_memory.html. Accessed: 2026-06-08.
- [30] Linus Schrage. 1968. Letter to the Editor—A Proof of the Optimality of the Shortest Remaining Processing Time Discipline. *Oper. Res.* 16, 3 (June 1968), 687–690. doi:10.1287/opre.16.3.687
- [31] SGLang Diffusion Team. 2025. SGLang Diffusion: Accelerating Video and Image Generation. <https://www.lmsys.org/blog/2025-11-07-sglang-diffusion/>. LMSYS Blog.
- [32] Aashaka Shah, Vijay Chidambaram, Meghan Cowan, Saeed Maleki, Madan Musuvathi, Todd Mytkowicz, Jacob Nelson, Olli Saarikivi, and Rachee Singh. 2023. TACCL: Guiding Collective Algorithm Synthesis using Communication Sketches. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, Boston, MA, 593–612. <https://www.usenix.org/conference/nsdi23/presentation/shah>
- [33] Min Si, Pavan Balaji, Yongzhou Chen, Ching-Hsiang Chu, Adi Gangidi, Saif Hasan, Subodh Iyengar, Dan Johnson, Bingzhe Liu, Regina Ren, Deep Shah, Ashmitha Jeevaraj Shetty, Greg Steinbrecher, Yulun Wang, Bruce Wu, Xinfeng Xie, Jingyi Yang, Mingran Yang, Kenny Yu, Minlan Yu, Cen Zhao, Wes Bland, Denis Boyda, Suman Gumudavelli, Prashanth Kannan, Cristian Lumezanu, Rui Miao, Zhe Qu, Venkat Ramesh, Maxim Samoylov, Jan Seidel, Srikanth Sundaresan, Feng Tian, Qiye Tan, Shuqiang Zhang, Yimeng Zhao, Shengbao Zheng, Art Zhu, and Hongyi Zeng. 2026. Collective Communication for 100k+ GPUs. arXiv:2510.20171 [cs.DC] <https://arxiv.org/abs/2510.20171>
- [34] Desen Sun, Zepeng Zhao, and Yuke Wang. 2025. PATCHEDSERVE: A Patch Management Framework for SLO-Optimized Hybrid Resolution Diffusion Serving. arXiv:2501.09253 [cs.DC] <https://arxiv.org/abs/2501.09253>
- [35] Desen Sun, Zepeng Zhao, and Yuke Wang. 2026. MixFusion: A Patch-Level Parallel Serving System for Mixed-Resolution Diffusion Models. In *Proceedings of the 31st ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming* (Sydney, NSW, Australia) (PPoPP '26). Association for Computing Machinery, New York, NY, USA, 522–536. doi:10.1145/3774934.3786420
- [36] Meituan LongCat Team, Xunliang Cai, Qilong Huang, Zhuoliang Kang, Hongyu Li, Shijun Liang, Liya Ma, Siyu Ren, Xiaoming Wei, Rixu Xie, and Tong Zhang. 2025. LongCat-Video Technical Report. arXiv:2510.22200 [cs.CV] <https://arxiv.org/abs/2510.22200>
- [37] Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages* (Phoenix, AZ, USA) (MAPL 2019). Association for Computing Machinery, New York, NY, USA, 10–19. doi:10.1145/3315508.3329973

- [38] Manjunath Gorentla Venkata, Valentine Petrov, Sergey Lebedev, Devendar Bureddy, Ferrol Aderholdt, Joshua Ladd, Gil Bloch, Mike Dubman, and Gilad Shainer. 2025. Unified Collective Communication: A Unified Library for CPU, GPU, and DPU Collectives. *IEEE Micro* 45, 2 (March 2025), 26–35. doi:10.1109/MM.2025.3534638
- [39] Team Wan, Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, Chen-Wei Xie, Di Chen, Feiwu Yu, Haiming Zhao, Jianxiao Yang, Jianyuan Zeng, Jiayu Wang, Jingfeng Zhang, Jingren Zhou, Jinkai Wang, Jixuan Chen, Kai Zhu, Kang Zhao, Keyu Yan, Lianghua Huang, Mengyang Feng, Ningyi Zhang, Pandeng Li, Pingyu Wu, Ruihang Chu, Ruili Feng, Shiwei Zhang, Siyang Sun, Tao Fang, Tianxing Wang, Tianyi Gui, Tingyu Weng, Tong Shen, Wei Lin, Wei Wang, Wei Wang, Wenmeng Zhou, Wenten Wang, Wenting Shen, Wenyuan Yu, Xianzhong Shi, Xiaoming Huang, Xin Xu, Yan Kou, Yangyu Lv, Yifei Li, Yijing Liu, Yiming Wang, Yingya Zhang, Yitong Huang, Yong Li, You Wu, Yu Liu, Yulin Pan, Yun Zheng, Yuntao Hong, Yupeng Shi, Yutong Feng, Zeyinzi Jiang, Zhen Han, Zhi-Fan Wu, and Ziyu Liu. 2025. Wan: Open and Advanced Large-Scale Video Generative Models. arXiv:2503.20314 [cs.CV] <https://arxiv.org/abs/2503.20314>
- [40] William Won, Midhilesh Elavazhagan, Sudarshan Srinivasan, Swati Gupta, and Tushar Krishna. 2024. TACOS: Topology-Aware Collective Algorithm Synthesizer for Distributed Machine Learning. In *Proceedings of the 2024 57th IEEE/ACM International Symposium on Microarchitecture (Austin, TX, USA) (MICRO '24)*. IEEE Press, 856–870. doi:10.1109/MICRO61859.2024.00068
- [41] Chenfei Wu, Jiahao Li, Jingren Zhou, Junyang Lin, Kaiyuan Gao, Kun Yan, Sheng ming Yin, Shuai Bai, Xiao Xu, Yilei Chen, Yuxiang Chen, Zecheng Tang, Zekai Zhang, Zhengyi Wang, An Yang, Bowen Yu, Chen Cheng, Dayiheng Liu, Deqing Li, Hang Zhang, Hao Meng, Hu Wei, Jingyuan Ni, Kai Chen, Kuan Cao, Liang Peng, Lin Qu, Minggang Wu, Peng Wang, Shuting Yu, Tingkun Wen, Wensen Feng, Xiaoxiao Xu, Yi Wang, Yichang Zhang, Yongqiang Zhu, Yujia Wu, Yuxuan Cai, and Zenan Liu. 2025. Qwen-Image Technical Report. arXiv:2508.02324 [cs.CV] <https://arxiv.org/abs/2508.02324>
- [42] Yifei Xia, Fangcheng Fu, Hao Yuan, Hanke Zhang, Xupeng Miao, Yijun Liu, Suhan Ling, Jie Jiang, and Bin Cui. 2025. TridentServe: A Stage-level Serving System for Diffusion Pipelines. arXiv:2510.02838 [cs.DC] <https://arxiv.org/abs/2510.02838>
- [43] Zhuoyi Yang, Jiayan Teng, Wendi Zheng, Ming Ding, Shiyu Huang, Jiazheng Xu, Yuanming Yang, Wenyi Hong, Xiaohan Zhang, Guanyu Feng, Da Yin, Yuxuan Zhang, Weihang Wang, Yean Cheng, Bin Xu, Xiaotao Gu, Yuxiao Dong, and Jie Tang. 2025. CogVideoX: Text-to-Video Diffusion Models with An Expert Transformer. arXiv:2408.06072 [cs.CV] <https://arxiv.org/abs/2408.06072>
- [44] Fanjiang Ye, Zhangke Li, Xinrui Zhong, Ethan Ma, Russell Chen, Kaijian Wang, Jingwei Zuo, Desen Sun, Ye Cao, Triston Cao, Myungjin Lee, Arvind Krishnamurthy, and Yuke Wang. 2026. GENSERVE: Efficient Co-Serving of Heterogeneous Diffusion Model Workloads. arXiv:2604.04335 [cs.DC] <https://arxiv.org/abs/2604.04335>
- [45] Peiqi Yin, Jiangyun Zhu, Han Gao, Chenguang Zheng, Yongxiang Huang, Taichang Zhou, Ruirui Yang, Weizhi Liu, Weiqing Chen, Canlin Guo, Didan Deng, Zifeng Mo, Cong Wang, James Cheng, Roger Wang, and Hongsheng Liu. 2026. vLLM-Omni: Fully Disaggregated Serving for Any-to-Any Multimodal Models. arXiv:2602.02204 [cs.DC] <https://arxiv.org/abs/2602.02204>
- [46] Zangwei Zheng, Xiangyu Peng, Yuxuan Lou, Chenhui Shen, Tom Young, Xinying Guo, Binluo Wang, Hang Xu, Hongxin Liu, Mingyan Jiang, Wenjun Li, Yuhui Wang, Anbang Ye, Gang Ren, Qianran Ma, Wanying Liang, Xiang Lian, Xiwen Wu, Yuting Zhong, Zhuangyan Li, Chaoyu Gong, Guojun Lei, Leijun Cheng, Limin Zhang, Minghao Li, Ruijie Zhang, Silan Hu, Shijie Huang, Xiaokang Wang, Yuanheng Zhao, Yuqi Wang, Ziang Wei, and Yang You. 2026. Open-Sora 2.0: Training a Commercial-Level Video Generation Model in \$200k. arXiv:2503.09642 [cs.GR] <https://arxiv.org/abs/2503.09642>
- [47] Zangwei Zheng, Xiangyu Peng, Tianji Yang, Chenhui Shen, Shenggui Li, Hongxin Liu, Yukun Zhou, Tianyi Li, and Yang You. 2024. Open-Sora: Democratizing Efficient Video Production for All. arXiv:2412.20404 [cs.CV] <https://arxiv.org/abs/2412.20404>
- [48] Yang Zhou, Zhongjie Chen, Ziming Mao, ChonLam Lao, Shuo Yang, Pravein Govindan Kannan, Jiaqi Gao, Yilong Zhao, Yongji Wu, Kaichao You, Fengyuan Ren, Zhiying Xu, Costin Raiciu, and Ion Stoica. 2025. An Extensible Software Transport Layer for GPU Networking. arXiv:2504.17307 [cs.NI] <https://arxiv.org/abs/2504.17307>
- [49] Yuan Zhou, Qiuyue Wang, Yuxuan Cai, and Huan Yang. 2024. Allegro: Open the Black Box of Commercial-Level Video Generation Model. arXiv:2410.15458 [cs.CV] <https://arxiv.org/abs/2410.15458>